Activity 10: Produce the Traceable Design and Release Record

Course: Build ASP.NET Web Apps with Vibe Coding (TGS-2023040477)

Alignment: LO5 · K5/A5

Duration: 55 minutes

Runnable snapshot: src/LocalCRM

Goal

Map user specifications to components, interface contracts, decisions and verification evidence.

What you will produce

C4-style diagrams, ADR, requirements traceability matrix, API contract hash and release checklist

Safety and data boundary

Use only the supplied fictional records in mock-data/. Do not paste personal, customer, credential, or production data into an AI tool. Review every generated diff before running it.

Vibe script

> Using requirements.md, component-map.md, openapi-v1.json and the Activity evidence, draft ADR-0001 and a traceability matrix. Every requirement must map to a component, interface or page, control, test/evidence and status. Mark gaps; do not invent evidence.

Step-by-step

Step 1 — Inventory authoritative inputs

List requirements, code components, OpenAPI contract, tests and evidence. Mark missing files before generating documentation.

Step 2 — Create a system context diagram

Show Staff User, LocalCRM, Azure App Service, database and monitoring as named trust/ownership boundaries.

Step 3 — Create a container diagram

Show Razor Pages UI, controller API, application service, EF Core adapter and SQLite/Azure SQL boundary.

Step 4 — Write ADR-0001

Record context, decision, at least two considered options, consequences and status. Use exact repository paths.

Step 5 — Build the traceability matrix

Use columns Requirement, Design component, Interface/control, Verification evidence, Status and Owner.

Step 6 — Hash the interface contract

Record the OpenAPI file hash beside the build SHA.

```
shasum -a 256 evidence/openapi-v1.json
git rev-parse HEAD
```

Step 7 — Close evidence gaps

Run only the missing checks. Never mark a row Pass from an AI claim or expected result alone.

Step 8 — Review with the stakeholder

Ask a peer to confirm the five LOs and REQ-001–004 are represented. Record reviewer/date and unresolved actions.

Step 9 — Package the release record

Create docs/release-v6.1.md listing artifact hash, configuration assumptions, known limitations, rollback and approval state.

Step 10 — Final gate

Run format, build, tests, secret scan, OpenAPI check and smoke test; attach command output to the release record.

```
dotnet format LocalCRM/LocalCRM.csproj --verify-no-changes
dotnet build LocalCRM/LocalCRM.csproj -c Release
dotnet test -c Release
git status --short
```

Expected evidence: All checks pass and the working tree contains only intentional documentation/evidence changes.

Run the supplied snapshot

The Activity folder already contains a complete stage-appropriate ASP.NET Core project and mock data. From this Activity folder:

```
cd src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5110
```

Open http://localhost:5110. For Activities 7–10, also probe http://localhost:5110/api/contacts; for Activities 8–10, probe /openapi/v1.json and /health/ready.

Run the supplied integration tests

From this Activity folder:

```
dotnet test src/LocalCRM.Tests/LocalCRM.Tests.csproj
```

Expected evidence: three tests pass for readiness, JSON collection output and invalid-input ProblemDetails.

Acceptance checklist

□ The project targets net10.0 and restores without an unreviewed package.

□ The app starts from this Activity folder without relying on another Activity.

□ The supplied mock contacts are fictional and load as described.

□ Positive and negative paths match the expected status or UI state.

□ The diff contains only intended files and no secret.

Troubleshooting

- NETSDK1045: install a .NET 10 SDK or select the installed 10.0 feature band in global.json.
- Address already in use: stop the previous Activity process or use --urls http://localhost:PORT with a free port.
- HTTP 404: compare the exact route, method, route constraint and Activity stage.

- HTTP 400/415: inspect ProblemDetails and confirm JSON shape plus Content-Type: application/json.
- SQLite failure (Activities 8–10): remove only the local training localcrm.db, then restart so the snapshot recreates mock data.

Reflection and evidence record

1. Explain how the evidence demonstrates LO5 and K5/A5.

2. Name one AI suggestion you rejected or changed and why.

3. Identify one negative path you tested and the observable result.

4. State the next improvement you would make without widening the approved scope.

Evidence item	Record the path, command result or observation
Build or test result	
Runtime URL and HTTP status	
Positive-path evidence	
Negative-path evidence	
Reviewed AI change or rejection	
Commit or evidence reference	